

1. Explain why having a standardized system development process is important to an organization.

CCCC E PQR

A standardized system development process is important for an organization because:

1. **Consistency:** It makes sure all projects follow the same steps, leading to predictable results.
2. **Efficiency:** Saves time and effort by using the same method for all projects.
3. **Quality Control:** Helps maintain high quality in all projects.
4. **Communication:** Makes it easier for teams to work together.
5. **Risk Management:** Helps identify and manage problems early.
6. **Project Tracking:** Makes it easier to track how the project is going.
7. **Cost Control:** Helps save money by reducing errors and rework.
8. **Compliance:** Ensures projects follow rules and regulations.

1. **ধারাবাহিকতা:** এটি নিশ্চিত করে যে সব প্রকল্প একই ধাপে চলে, ফলে পূর্বানুমানযোগ্য ফলাফল পাওয়া যায়।
2. **দক্ষতা:** একই পদ্ধতি ব্যবহার করে কাজ করায় সময় ও শ্রম সাশ্রয় হয়।
3. **গুণমান নিয়ন্ত্রণ:** সব প্রকল্পে উচ্চ মান বজায় রাখতে সাহায্য করে।
4. **যোগাযোগ:** টিমের মধ্যে কাজ করা সহজ হয়।
5. **ঝুঁকি ব্যবস্থাপনা:** সমস্যা আগে থেকেই চিহ্নিত ও মোকাবিলা করা যায়।
6. **প্রকল্প ট্র্যাকিং:** প্রকল্প কীভাবে চলছে তা সহজে পর্যবেক্ষণ করা যায়।
7. **ব্যয় নিয়ন্ত্রণ:** ভুল ও পুনরায় কাজ কমিয়ে খরচ সাশ্রয় হয়।
8. **নিয়ম মেনে চলা:** প্রকল্পগুলো নিয়ম ও বিধি অনুসরণ করছে তা নিশ্চিত করে।

2.How are system life cycle and system development methodology related?

System Life Cycle and System Development Methodology are closely related concepts in the field of information systems.

Relationship:

1. **System Life Cycle** refers to the overall **stages** a system goes through from planning to release (e.g., planning → analysis → design → development → testing → implementation → maintenance).
2. **System Development Methodology** is the **structured approach or method** used to carry out each of these life cycle stages (e.g., Waterfall, Agile, Spiral).
3. **Methodology guides** how the life cycle stages are done — for example:
 - In **Waterfall**, stages are done one after another.
 - In **Agile**, stages are repeated in short cycles (iterations).

Example of System Life Cycle and Methodology Relationship:

- Suppose a company is developing a **library management system**.

System Life Cycle Steps (What to Do):

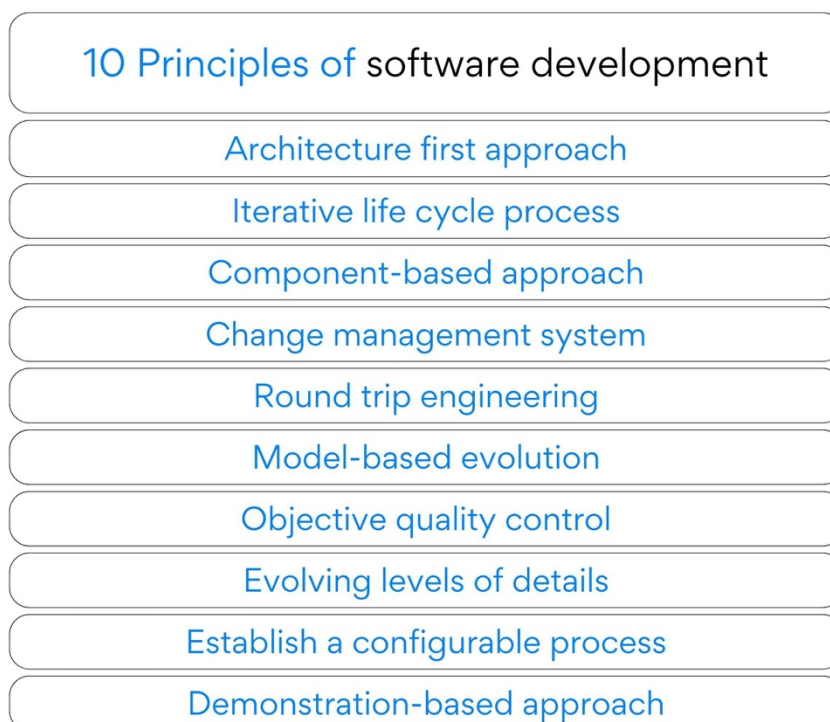
1. **Planning** – Decide to create the system.
2. **Analysis** –gather requirement
3. **Design** – Requirement → visualization/design
4. **Development** – visualization/design → coding → development
5. **Testing** –then testing and gather feedback to users.
6. **Implementation** –solve the problem and implementation.
7. **Maintenance** – Fix problems after launching.

System Development Methodology (How to Do It):

- The company uses the **Waterfall Model** to follow these **steps one after another**.
- Or they may use **Agile Method** to develop in **small parts and improve step by step**.

◆ So, the **Life Cycle** gives the overall process, and the **Methodology** gives the way to carry it out.

3.What are the 10 underlying principles for systems development?



✅ 10 Principles of Systems Development

Principle (English)	বাংলা সহজ ব্যাখ্যা
Development is a process	এটি ধাপে ধাপে হয় (প্ল্যান, ডিজাইন, টেস্ট)।
Built for a purpose	সিস্টেম কোনো কাজ বা সমস্যা সমাধানে হয়।
System = inter-related parts	সব অংশ একসাথে কাজ করে।
Uses a structured approach	একটি নির্দিষ্ট পদ্ধতিতে তৈরি হয় (যেমন Agile)।
Teamwork is required	টিমে কাজ করে সিস্টেম তৈরি হয়।
User involvement is key	ব্যবহারকারীর মতামত নেওয়া জরুরি।
Needs documentation	প্রতিটি ধাপ লিখে রাখা হয়।
Should be maintainable	ভবিষ্যতে সহজে ঠিক করা যায় এমন হতে হবে।
Focus on quality	নির্ভুল, দ্রুত ও ব্যবহার-বান্ধব হওয়া দরকার।
Ensure ethics and security	তথ্য নিরাপদ ও সঠিকভাবে ব্যবহার করতে হয়।

4. Why is documentation important throughout the development process?

Documentation is important in the development process because:

1. **Clear Communication:** It helps everyone understand the goals, design, and progress of the project.
2. **Record Keeping:** It keeps a record of decisions, designs, and changes for future reference.
3. **Consistency:** Ensures everyone follows the same standards and processes.
4. **Knowledge Transfer:** New team members can understand the project easily.
5. **Compliance:** It is often required by law to meet certain standards.
6. **Issue Resolution:** Helps solve problems by providing details about design and testing.
7. **Quality Control:** Ensures the system is of good quality by keeping track of requirements and feedback.
- 8.

5. Why are process management and project management necessary? (Important)

Process management and **project management** are necessary for several reasons:

Process Management: (CC-EI)

1. **Consistency:** It ensures that tasks are done the same way every time, leading to predictable and high-quality outcomes.
2. **Efficiency:** Helps streamline processes, reducing waste and improving productivity.
3. **Improvement:** Allows for continuous monitoring and refinement of processes for better performance.
4. **Control:** Helps identify and fix issues early to avoid larger problems later.

Project Management: (ORRG)

1. **Organization:** Helps plan, execute, and complete projects efficiently, ensuring all tasks are on schedule.
2. **Resource Allocation:** Ensures the right resources (**time, budget, manpower**) are used effectively.
3. **Risk Management:** Project risks are identified in advance and addressed..
4. **Goal Achievement:** Keeps the team focused on achieving the project goals within the set time and budget.

Both are crucial for successfully completing projects on time, within budget, and at the desired level of quality.

6.What is risk management? Why is it necessary? (Important)

Risk management is a process where potential risks are identified, analyzed, and evaluated, then steps are taken to control those risks.

Steps in Risk Management:

1. **Identify Risks:** Recognize all potential risks.
2. **Assess Risks:** Analyze the impact and risk.
3. **Analyze Risks:** Understand the causes and effects of the risks.
4. **Plan for Risks:** Decide on steps to reduce or avoid risks.
5. **control Risks:** Implement the plan to handle risks.
6. **Monitor Risks:** Regularly track the progress and status of risks.
7. **Document Risks:** Record all identified risks and actions taken.

7.Which stakeholders initiate most projects? What is the impetus for most projects?

Answer:

Most projects are initiated by **senior managers, business executives, or department heads** who identify a need for improvement or a business opportunity.

বাংলা:

বেশিরভাগ প্রকল্প শুরু করেন **উচ্চপদস্থ ম্যানেজার, ব্যবসা নির্বাহী বা বিভাগের প্রধানরা**, যারা কোনো উন্নয়ন প্রয়োজন বা ব্যবসায়িক সুযোগ খুঁজে পান।

What is the impetus for most projects?

Answer:

The main impetus for most projects is **to solve a problem, improve efficiency, reduce cost, or take advantage of new opportunities or technologies.**

বাংলা:

প্রকল্প শুরু হওয়ার মূল কারণ হলো **সমস্যা সমাধান, দক্ষতা বৃদ্ধি, খরচ কমানো অথবা নতুন সুযোগ বা প্রযুক্তি ব্যবহার করা।**

8. Who are the main participants in the scope definition? What are their goals in the scope definition?

Main Participants in Scope Definition:

1. **Project Sponsor** – টাকা দেয়, চায় কাজ যেন সফল হয়।
2. **Project Manager** – কাজ কিভাবে চলবে সেটার পরিকল্পনা করে।
3. **Clients / Customers** – যাদের জন্য সিস্টেম বানানো হচ্ছে, তারা কী চায় তা জানায়।
4. **Business Analysts** – গ্রাহকের চাহিদা বুঝে, সেটা ডকুমেন্ট করে।
5. **Technical Team (Developers/Architects)** – জানে কোন কাজ বাস্তবভাবে করা যাবে, কোনটা কঠিন।
6. **End Users** – যারা সিস্টেম ব্যবহার করবে, তারা ব্যবহারযোগ্যতা (usability) বোঝায়।

Goals in Scope Definition:

1. **Clear Objectives** – প্রকল্পে ঠিক কী হবে, সেটা বোঝা।
2. **Requirements Understanding** – ক্লায়েন্ট কী চায়, সেটা পরিষ্কারভাবে জানা।
3. **Avoid Scope Creep** – পরে যেন অতিরিক্ত কাজ না আসে, সেটা ঠেকানো।
4. **Set Boundaries** – কোন কাজ এই প্রকল্পে থাকবে, কোনটা থাকবে না – সেটা নির্ধারণ করা।
5. **Resource Planning** – কত সময়, লোক, ও টাকা লাগবে, সেটার প্ল্যান করা।

9. What are the three most important deliverables in scope definition?

1. **Project Scope Statement**
 - Describes what the project will deliver and what is outside its boundaries.
2. **Requirements Document**
 - Lists what the users and stakeholders need from the system.
3. **Work Breakdown Structure (WBS)**
 - Breaks the entire project into smaller, manageable tasks.

■ বাংলায় সহজভাবে:

1. **প্রজেক্ট স্কোপ স্টেটমেন্ট** – প্রজেক্টে কী হবে আর কী হবে না, তা স্পষ্ট করে লেখা।
2. **রিকোয়ারমেন্ট ডকুমেন্ট** – ইউজাররা কী চায় সেটা তালিকা করে লেখা।
3. **ওয়ার্ক ব্রেকডাউন স্ট্রাকচার (WBS)** – পুরো কাজ ছোট ছোট টাস্কে ভাগ করে লেখা।

বোঝার জন্য উদাহরণ:

1. **✓ Project Scope Statement**
→ প্রজেক্টে কী থাকবে, কী থাকবে না তা নির্ধারণ করে।
উদাহরণ: অনলাইন বই বিক্রি হবে, হোম ডেলিভারি নয়।
2. **✓ Requirements Document**
→ ইউজার কী কী চাইছে, তার লিস্ট।
উদাহরণ: সাইনআপ, সার্চ, পেমেন্ট সিস্টেম।
3. **✓ Work Breakdown Structure (WBS)**
→ পুরো কাজ ছোট ছোট ভাগে ভাগ করা।
উদাহরণ: ডিজাইন → ফাংশন → পেমেন্ট গেটওয়ে।

10. Who are the main participants in the requirements analysis phase? Why are they the main participants?

The main participants are:

1. **System Users**
2. **System Owners**
3. **System Analysts**
4. **Developers**
5. **QA (Quality Assurance) Team**

Why Are They the Main Participants?

1. **System Users:** They use the system, so they know what features and functions are needed.
2. **System Owners:** They set business goals and provide money and resources.
3. **System Analysts:** They collect and organize all the requirements clearly.
4. **Developers:** They check if the system can be built and suggest technical solutions.
5. **QA Team:** They make sure the requirements can be tested and that the system will work properly.

11. What feasibility analyses are made in the decision analysis? (T E L O S)

1. **Technical Feasibility**
→ Checks if the technology we need is available and usable.
2. **Economic Feasibility**
→ Checks if the project will give more benefit than cost.
3. **Legal Feasibility**
→ Checks if the project follows all laws, rules, or agreements.

4. **Operational Feasibility**
→ Checks if the system will work properly in real life and users will use it.
 5. **Schedule Feasibility**
→ Checks if the project can be finished on time.
-

বাংলা সারাংশ:

1. **Technical Feasibility** – দরকারি প্রযুক্তি আছে কি না ও ব্যবহারযোগ্য কি না, তা দেখা হয়।
2. **Economic Feasibility** – লাভ খরচের চেয়ে বেশি হবে কি না, তা দেখা হয়।
3. **Legal Feasibility** – আইন, নিয়ম বা চুক্তি মানা হচ্ছে কি না, তা দেখা হয়।
4. **Operational Feasibility** – সিস্টেম বাস্তবে কাজ করবে ও মানুষ ব্যবহার করবে কি না, তা দেখা হয়।
5. **Schedule Feasibility** – প্রজেক্ট সময়মতো শেষ করা যাবে কি না, তা দেখা হয়।

12. What is model-driven development? (Important)

Model-Driven Development (MDD) is a method where diagrams or models are first created, and then code is written accordingly.

- This makes it easier to understand and plan projects.
- Code can also be generated automatically from models

Model-Driven Development (MDD) হলো একটি পদ্ধতি যেখানে আগে **ডায়াগ্রাম বা মডেল** তৈরি করা হয়, তারপর সেই অনুযায়ী কোড লেখা হয়।

- এতে প্রজেক্ট বুঝতে ও পরিকল্পনা করতে সহজ হয়।
- মডেল থেকে স্বয়ংক্রিয়ভাবে কোডও তৈরি করা যায়।

13. Why is model-driven development popular?

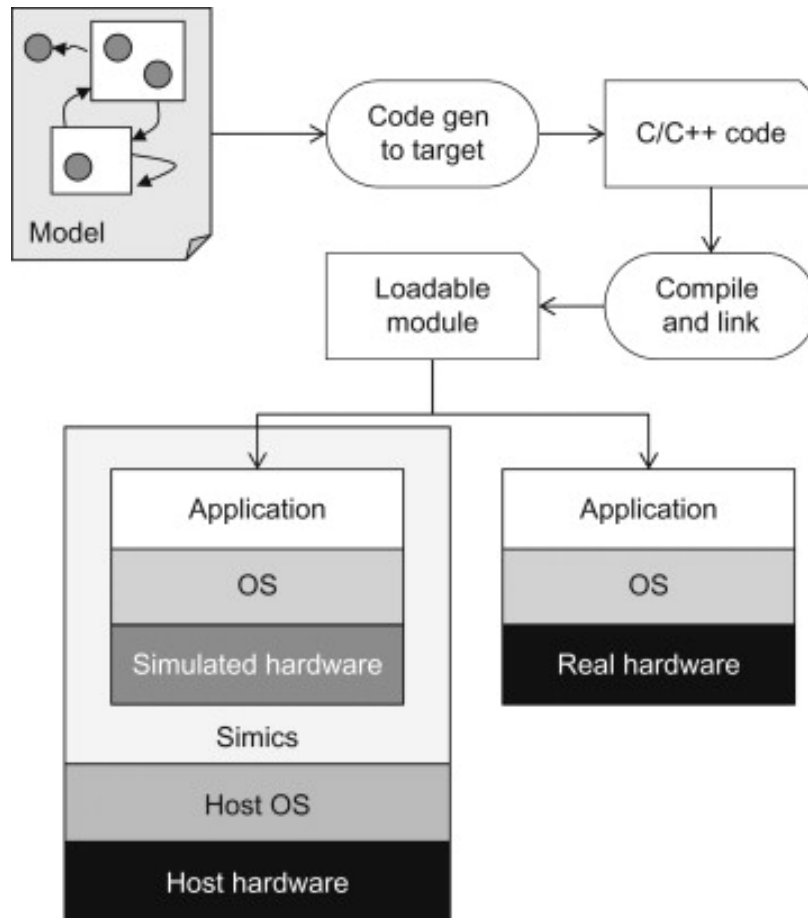
Model-Driven Development (MDD) is popular because it makes software development easier, faster, and more organized.

Reasons Why MDD is Popular:

1. **Better Understanding**
 - Models help teams **visualize the system** clearly before writing code.
2. **Improves Communication**
 - Easy to explain ideas to both technical and non-technical people using diagrams.
3. **Faster Development**
 - Code can be **automatically generated** from models, saving time.
4. **Reduces Errors**
 - Clear planning and structure help avoid mistakes in coding.
5. **Easier Maintenance**
 - Models make it simple to **update or change** the system later.

Model-Driven Development জনপ্রিয় কারণ:

1. **ভালোভাবে বোঝা যায়** – মডেল দেখে পুরো সিস্টেম আগে থেকে বোঝা যায়।
2. **যোগাযোগ সহজ হয়** – ডায়াগ্রাম দিয়ে সবাইকে সহজে বোঝানো যায়।
3. **দ্রুত কাজ হয়** – মডেল থেকে কোড বানানো যায়, সময় বাঁচে।
4. **ভুল কম হয়** – পরিকল্পনা ঠিক থাকলে কোডিং-এ ভুল কম হয়।
5. **পরিবর্তন সহজ** – ভবিষ্যতে কিছু বদলাতে হলে মডেল দেখে সহজে করা যায়।



14. What is rapid application development (RAD)?

Rapid Application Development (RAD) is a software development method that focuses on **fast development and quick delivery** of applications with the help of **user feedback**, **prototypes**, and **reusable components**.

Rapid Application Development (RAD) is a software development method that focuses on **fast development and quick delivery** of applications with the help of **user feedback**, **prototypes**, and **reusable components**.

◆ Key Features of RAD:

1. **Fast Development** – Projects are completed quickly.
2. **Prototyping** – Early versions of the system (prototypes) are built and improved with user feedback.
3. **User Involvement** – Users are involved at every stage to ensure the system meets their needs.
4. **Reusable Components** – Existing parts are reused to save time.
5. **Flexibility** – Changes can be made easily during development.

Rapid Application Development (RAD) হলো এমন এক ডেভেলপমেন্ট পদ্ধতি, যেখানে **দ্রুত** সফটওয়্যার তৈরি করা হয় **ইউজার ফিডব্যাক**, **প্রোটোটাইপ**, আর **পুরাতন কম্পোনেন্ট** ব্যবহার করে।

RAD এর বৈশিষ্ট্য:

1. **দ্রুত কাজ হয়**
2. **প্রোটোটাইপ বানানো হয়**
3. **ইউজারদের অংশগ্রহণ থাকে**
4. **আগে তৈরি জিনিস ব্যবহার করা হয়**
5. **সহজে পরিবর্তন করা যায়**

Benefits of RAD:

1. **Faster Delivery**
→ Software is developed and delivered quickly.
2. **User Satisfaction**
→ Continuous user feedback ensures the system meets their needs.
3. **Flexibility**
→ Easy to make changes during development.
4. **Early Detection of Errors**
→ Prototypes help find problems early.
5. **Better Communication**
→ Regular interaction between developers and users.
6. **Reduced Development Cost**
→ Reusing components saves time and money.
7. **High-Quality System**
→ More testing and feedback result in fewer bugs.

1. দ্রুত সফটওয়্যার ডেলিভারি হয়
2. ইউজার খুশি থাকে, কারণ তাদের মতামত নেয়া হয়
3. সহজে পরিবর্তন করা যায়
4. প্রথমেই ভুল ধরা যায়
5. ডেভেলপার আর ইউজারের মধ্যে ভালো যোগাযোগ হয়
6. কম খরচে ডেভেলপমেন্ট সম্ভব
7. ভালো মানের সফটওয়্যার পাওয়া যায়

16.What is computer assisted software engineering (CASE)? List some examples of CASE.

Computer-Assisted Software Engineering (CASE) is the use of software tools to assist in various stages of software development.

Benefits of CASE:

- **Increased Productivity:** Automates repetitive tasks, saving time.
- **Improved Quality:** Helps ensure consistency and standardization in the development process.
- **Better Documentation:** Facilitates easier documentation generation.
- **Easier Maintenance:** Provides tools for easier system modifications and updates.

Types of CASE Tools:

1. **Upper CASE Tools:**
 - Used for the early stages of software development, like requirements gathering, analysis, and design.
 - Examples:
 - **Rational Rose**

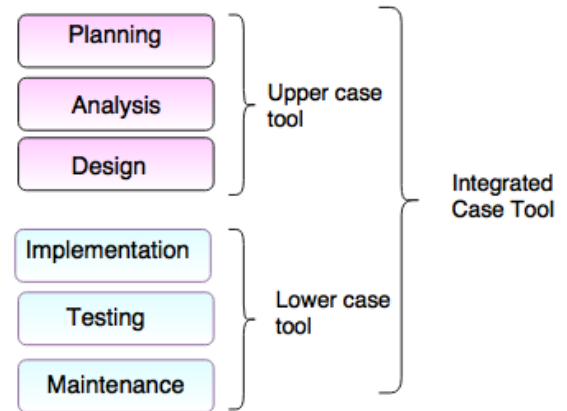
- **Microsoft Visio**

2. **Lower CASE Tools:**

- Used for coding, testing, and maintenance.
- Examples:
 - **Eclipse**
 - **JDeveloper**

3. **Integrated CASE Tools:**

- Combines both upper and lower CASE tools.
- Examples:
 - **Oracle JDeveloper**
 - **IBM Rational Suite**



Computer-Assisted Software Engineering (CASE) হল সফটওয়্যার ডেভেলপমেন্টের বিভিন্ন ধাপে সাহায্য করার জন্য সফটওয়্যার টুলস ব্যবহার। এগুলি বিভিন্ন পর্যায়ে সাহায্য করে, যেমন:

1. **Upper CASE Tools** – যেমন **Rational Rose**, **Microsoft Visio**, যা ডেভেলপমেন্টের প্রথম দিকের কাজগুলোর জন্য ব্যবহার হয়।
2. **Lower CASE Tools** – যেমন **Eclipse**, **JDeveloper**, যা কোডিং এবং টেস্টিংয়ের জন্য ব্যবহৃত হয়।
3. **Integrated CASE Tools** – যেমন **Oracle JDeveloper**, **IBM Rational Suite**, যা উল্লিখিত দুই ধরনের কাজের জন্য একত্রে ব্যবহৃত হয়।

IBM Rational Rose – UML সেলফ সিলেম মলড তৈরি কিলৈ েযহ্নৈ হয়।

Oracle Designer – যডোলেি ও সেজলেি প্রলিি সভজোইলেি জেয।

Microsoft Visio – যলোোোেত , DFD ও ERD ডোযোগ্রোম আঁকলৈ েযহ্নৈে।

CA ERwin – যডোলেি সভজোইলেি জেয জেসপ্রয েুু ।

Enterprise Architect – সিলেম মলড ও অযোোো েইসিলিি জেয।

Visible Analyst – ডোযোগ্রোম সেলফ সিলেম সভজোইলেি জেয।

PowerDesigner (SAP) – যডো ও প্রলিি মলডস িংলযি জেয েযহ্নৈে।

The Capability Maturity Model (CMM) was developed by the Software Engineering Institute at Carnegie Mellon, and is widely used by both the private and public sectors. What is the purpose of the CMM framework and how does it achieve this?

Purpose of CMM:

The main purpose of the CMM (Capability Maturity Model) is to help organizations improve the way they develop software. It gives a clear path to move from a basic, disorganized process to a more structured and efficient one.

CMM is a step-by-step model, with 5 phases. Each phase makes the organization's work more planned, controlled, and effective.

CMM এর উদ্দেশ্য কী?

CMM (Capability Maturity Model)-এর মূল উদ্দেশ্য হলো, কোনো কোম্পানি কীভাবে সফটওয়্যার বানায়, সেটাকে ধাপে ধাপে উন্নত করতে সাহায্য করা।

কীভাবে কাজ করে?

CMM একটি ধাপে ধাপে মডেল, যার ৫টি পর্যায় আছে। প্রতিটি ধাপ প্রতিষ্ঠানের কাজকে আরও পরিকল্পিত, নিয়ন্ত্রিত, এবং কার্যকর করে তোলে।

1. Initial (Chaotic)

- **No defined process.** Everything is unplanned and unpredictable.
- Success depends on individual efforts.

2. Repeatable

- **Basic processes** like project planning and tracking are established.
- Successes can be **repeated** on similar projects.

3. Defined

- Processes are **standardized and documented**.
- A **common organization-wide** process is used.

4. Managed

- Processes are **measured and controlled**.
- Uses **data and metrics** to track performance.

5. Optimizing

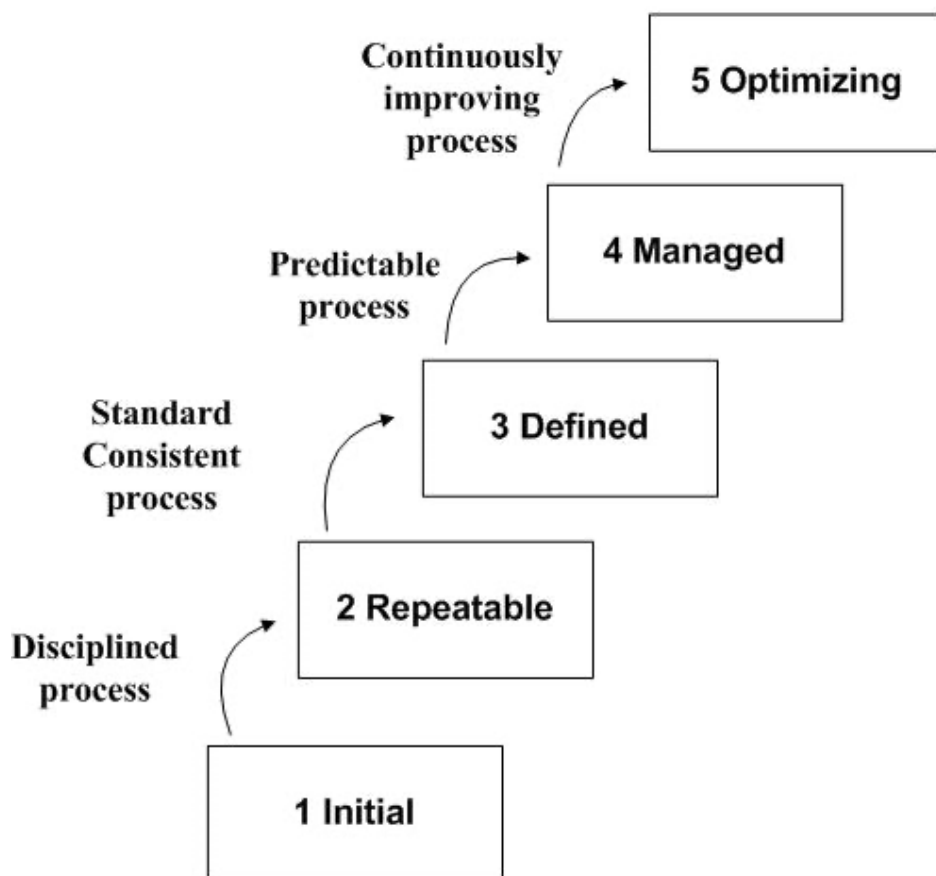
- Focus on **continuous improvement**.

- Uses feedback and innovation to make processes better.

বাংলায় সংক্ষেপে:

1. **Initial** – কোন নিয়ম নেই, অগোছালো।
2. **Repeatable** – কিছু নিয়ম আছে, আগের সাফল্য আবার করা যায়।
3. **Defined** – সব নিয়ম লিখিত ও সবার জন্য এক।
4. **Managed** – নিয়ম অনুযায়ী কাজ হয়, মাপা হয়।
5. **Optimizing** – নিয়ম সব আছে, এখন আরও ভালো করার চেষ্টা চলে।

Five Levels of Software Process Maturity in CMM



◆ 1. Initial (অগোছালো ধাপ)

ভাবো: তুমি হঠাৎ ৫ জন বন্ধু মিলে একটা সফটওয়্যার প্রজেক্ট শুরু করলা।

- কেউ ডকুমেন্ট লেখে না,
 - কেউ কি কাজ করবে ঠিক নাই,
 - আজ যেটা করলা, কাল ভুলে গেলে আবার নতুন করে শুরু।
- 🔴 **উদাহরণ:** যেমন, টিউশন পড়ানোর অ্যাপ বানাতে গিয়ে কেউ লগইন বানালা, কেউ ডাটাবেইজ ছাড়া কোড লিখে ফেলে।
-

◆ 2. Repeatable (পুনরাবৃত্তিযোগ্য ধাপ)

ভাবো: তোমরা একটা প্রজেক্ট ভালো করলা, তারপর সেটার নিয়মগুলো মনে রেখে আবার অন্য প্রজেক্টে করলা।

- কিছু নিয়ম তৈরি হয়েছে, কিন্তু সব কাজের জন্য না।
- 🔴 **উদাহরণ:** প্রথম প্রজেক্টে GitHub ব্যবহার করে দেখলা ভালো হয়েছে, পরের প্রজেক্টেও তাই ব্যবহার করলা।
-

◆ 3. Defined (সংজ্ঞায়িত ধাপ)

ভাবো: এখন তোমাদের টিমে একটা রুলবুক আছে—

- কে কখন কোন টুলে কোড করবে,
 - কোন ফরম্যাটে রিপোর্ট দিবে,
 - সব কাজের লিখিত নিয়ম আছে।
- 🔴 **উদাহরণ:** তোমার টিম Trello দিয়ে টাস্ক ম্যানেজ করে, ফিক্সড ফরম্যাটে ডকুমেন্টেশন করে, কোড স্টাইল ফলো করে।
-

◆ 4. Managed (পরিমাপযোগ্য ধাপ)

ভাবো: এখন তোমরা শুধু নিয়ম মানো না, বরং প্রতিটি কাজ পরিমাপ করো।

- কত সময় লাগলো,
- কয়টা বাগ ধরা পড়লো,

- টিম কতটা প্রোডাক্টিভ — সব হিসেব রাখা হয়।
 ✖ **উদাহরণ:** তুমি জানো যে, একেকটা ফিচার বানাতে ৩ দিন লাগে, তাই সেই হিসেব ধরে টাইমলাইন ঠিক করো।

◆ 5. Optimizing (উন্নয়নমুখী ধাপ)

ভাবো: এখন টিম চিন্তা করে, "আরও ভালো করা যায় না?"

- নতুন টুল নিয়ে আসে,
- অটোমেশন করে,
- টিম মেম্বারদের ফিডব্যাক নেয়।
 ✖ **উদাহরণ:** তুমি দেখে ফেলো যে কোড ম্যানুয়ালি রিভিউ করতে সময় লাগে, তাই কোড লিন্টিং টুল বসিয়ে দিলে।

3. Table 3-1 in the textbook illustrates the difference in a typical project's duration, person-months, quality, and cost, depending upon whether an organization's system development process is at CMM level 1, 2, or 3. Between which two CMM levels does an organization gain the greatest benefit in terms of percentage of improvement? What do you think is the reason for this?

TABLE 3 - 1 Impact of System Development "Process" on Quality

CMM Project Statistics for a Project Resulting in 200,000 Lines of Code

Organization's CMM Level	Project Duration (months)	Project Person-Months	Number of Defects Shipped	Median Cost (\$ millions)	Lowest Cost (\$ millions)	Highest Cost (\$ millions)
1	30	600	61	5.5	1.8	100+
2	18.5	143	12	1.3	.96	1.7
3	15	80	7	.728	.518	.933

Source: Master Systems, Inc.

✓ Greatest Benefit?

Let's compare **Level 1 → 2** and **Level 2 → 3** in terms of **cost** and **person-months**:

◆ *From Level 1 to 2:*

- **Cost drops from 5.5M → 1.3M ⇒**
Improvement = $((5.5-1.3)/5.5) \times 100$ $((5.5 - 1.3) / 5.5) \times 100\% \approx 76.36\%$
- **Person-months drops from 600 → 143 ⇒**
Improvement = $((600-143)/600) \times 100$ $((600 - 143) / 600) \times 100\% \approx 76.17\%$

◆ *From Level 2 to 3:*

- **Cost drops from 1.3M → 0.728M ⇒**
Improvement = $((1.3-0.728)/1.3) \times 100$ $((1.3 - 0.728) / 1.3) \times 100\% \approx 44.0\%$
- **Person-months drops from 143 → 80 ⇒**
Improvement = $((143-80)/143) \times 100$ $((143 - 80) / 143) \times 100\% \approx 44.05\%$

🏆 **Conclusion:**

The **greatest benefit** in terms of **percentage improvement** comes between:

🔥 **CMM Level 1 → Level 2**

🤔 **Why is the improvement so big between Level 1 and 2?**

Because **Level 1** is chaotic — there's **no defined process**, people work ad-hoc, and mistakes are common.

Moving to **Level 2** introduces:

- **Basic project management,**
- **Repeatable processes,**
- **Better planning,**
- **Defect reduction,**
- **And more control.**

4. Systems development methodology and system life cycle are two terms that are frequently used and just as frequently misused. What is the difference between the two terms?

✓ System Life Cycle

1. It is a **step-by-step process** of developing a system.
2. It shows the steps a system goes through from start to finish. একটি সিস্টেম শুরু থেকে শেষ পর্যন্ত কোন কোন ধাপ অতিক্রম করে তা দেখায়।
3. Common stages: Planning → Analysis → Design → Implementation → Maintenance
4. It tells **what to do** at each phase.
5. It applies to **all types of system projects**.

✓ System Development Methodology

1. It is a **method of doing the work**.
2. It tells **how to do the tasks** in each life cycle phase.
3. Examples: Waterfall, Agile, Spiral, RAD
4. It defines the **rules, tools, and techniques** used.
5. Helps to choose the **best way to complete the project successfully**.

5. Describe how using a systems development methodology is in line with CMM goals and can help an organization increase its maturity level.

Answer:

1. **Standard Process:** Follows a consistent approach, aligning with CMM's goal for consistency (Level 2 & 3).
2. **Better Project Management:** Helps track time, costs, and progress, supporting CMM's project control focus.
3. **Documentation:** Ensures tracking of improvements, meeting CMM requirements.
4. **Improved Quality:** Regular testing and reviews enhance quality, aligning with CMM's quality focus.
5. **Continuous Improvement:** Lessons learned improve future performance, advancing CMM maturity.

In short: A methodology provides consistency, control, and quality, helping to improve and advance through CMM levels.

□ **স্ট্যান্ডার্ড প্রক্রিয়া** : একটি নির্দিষ্ট পদ্ধতি অনুসরণ করলে নিশ্চিত হয় যে সবাই একইভাবে কাজ করে, যা CMM-এর ধারাবাহিকতার লক্ষ্যের (স্তর 2 এবং 3) সাথে সামঞ্জস্যপূর্ণ।

□ **উন্নত প্রকল্প ব্যবস্থাপনা** : সময়, খরচ এবং অগ্রগতি ট্র্যাক করা সহজ, যা প্রকল্প নিয়ন্ত্রণের উপর CMM-এর ফোকাসের সাথে সামঞ্জস্যপূর্ণ।

- **ডকুমেন্টেশন** : পদ্ধতির জন্য ডকুমেন্টেশন প্রয়োজন, যা উন্নতি ট্র্যাক করতে এবং CMM-এর প্রয়োজনীয়তা পূরণে সহায়তা করে।
- **উন্নত মান** : নিয়মিত পর্যালোচনা এবং পদ্ধতিগত পরীক্ষা সফটওয়্যারের মান উন্নত করে, যা CMM-এর মানের উপর মনোযোগের সাথে সঙ্গতিপূর্ণ।
- **ক্রমাগত উন্নতি** : অতীতের ভুল থেকে শিক্ষা গ্রহণের ফলে ভবিষ্যতের কর্মক্ষমতা আরও ভালো হয়, যা CMM পরিপক্বতার মাত্রা বৃদ্ধিতে সহায়তা করে।

6. A number of underlying principles are common to all systems development methodologies. Identify these underlying principles and explain them.

1. **Involve Users** – Keep users involved from the beginning.
2. **Problem-Solving** – Identify problems and solve them.
3. **Phased Activities** – Do the work step by step.
4. **Documentation** – Write everything down.
5. **Set Standards** – Follow specific rules.
6. **Manage Process** – Manage time, costs, and work properly.
7. **Investment View** – Think of the project as a profitable investment.
8. **Be Flexible** – Be prepared for change.
9. **Divide and Conquer** – Do big tasks by dividing them.
10. **Design for Change** – Design for the future.

7. The PIECES framework was developed by James Wetherbe as a means to classify problems. Identify the categories, then categorize the following problems using the PIECES framework:

- a. Duplicate data is stored throughout the system.
- b. There is a need to port an existing application to PDA devices.
- c. Quarterly sales reports need to be generated automatically.
- d. Employees can gain access to confidential portions of the personnel system.
- e. User interfaces for the inventory system are difficult and confusing, resulting in a high frequency of incorrect orders.

PIECES Framework এর ছয়টি ক্যাটাগরি:

1. **P – Performance** (ফ্রুত কাজ হয় কিনা)
2. **I – Information** (তথ্য সঠিক ও যথেষ্ট কিনা)
3. **E – Economics** (লাভ-ক্ষতি, খরচ বেশি হচ্ছে কিনা)
4. **C – Control** (নিয়ন্ত্রণ ঠিক আছে কিনা, নিরাপত্তা ইস্যু আছে কিনা)
5. **E – Efficiency** (সিস্টেম ভালোভাবে সম্পদ ব্যবহার করছে কিনা)
6. **S – Service** (ব্যবহারকারীর সুবিধা ও সন্তুষ্টি)

- a. Duplicate data → **Efficiency**
- b. Port to PDA → **Service**
- c. Auto sales reports → **Information**
- d. Unauthorized access → **Control**
- e. Confusing UI, wrong orders → **Service**

8. Each phase of a project includes specific deliverables that must be produced and delivered to the next phase. Using the textbook's hypothetical FAST methodology, what are the deliverables for the requirements analysis, logical design, and physical design/integration phases?

1. Requirements Analysis (চাহিদা বিশ্লেষণ)

- **What happens:** User কী চায়, সেটা collect করা
- **Deliverables:**
 - Requirements List (চাহিদার তালিকা)
 - Feasibility Report (সম্ভাব্যতা বিশ্লেষণ)
- **Example:** A restaurant system needs to take orders, print bills, and manage the menu.

2. Logical Design (লজিক্যাল ডিজাইন)

- **What happens:** System কীভাবে কাজ করবে, সেটা diagram এ plan করা
- **Deliverables:**
 - DFD (Data Flow Diagram)
 - ER Diagram (Entity Relationship)
- **Example:** Customer → Order → Kitchen → Bill → Payment (flow diagram format)

3. Physical Design & Integration (বাস্তব ডিজাইন ও সংযোগ)

- **What happens:** Actual coding, table creation, এবং system গুলো connect করা
- **Deliverables:**
 - Code (প্রোগ্রাম কোড)
 - Database Tables
 - Integration Report
- **Example:** Order-taking system coded in Python, database in MySQL.

9. Scope definition is the first phase of the FAST methodology, and it is either the first phase or part of the first phase of most methodologies. What triggers the scope phase, which stakeholders are involved in this phase, what two essential questions need to be answered, and what three important deliverables come out of this phase?

◆ **Phase Name:** Scope Definition Phase (পরিধি নির্ধারণ ধাপ)

✓ What Triggers It?

This phase begins when a problem is detected or a new system is needed.

বাংলায়: যখন কোনো সমস্যা ধরা পড়ে বা নতুন সিস্টেমের দরকার হয়, তখন এই ধাপ শুরু হয়।

Stakeholders Involved

- **System owners** (যারা সিস্টেমের জন্য টাকা দিচ্ছে বা মালিক)
- **System users** (যারা সিস্টেম ব্যবহার করবে)
- **System analysts** (যারা সিস্টেম তৈরি করবে)

? Two Essential Questions

1. **What problems or opportunities exist?**
2. **What should be done?**

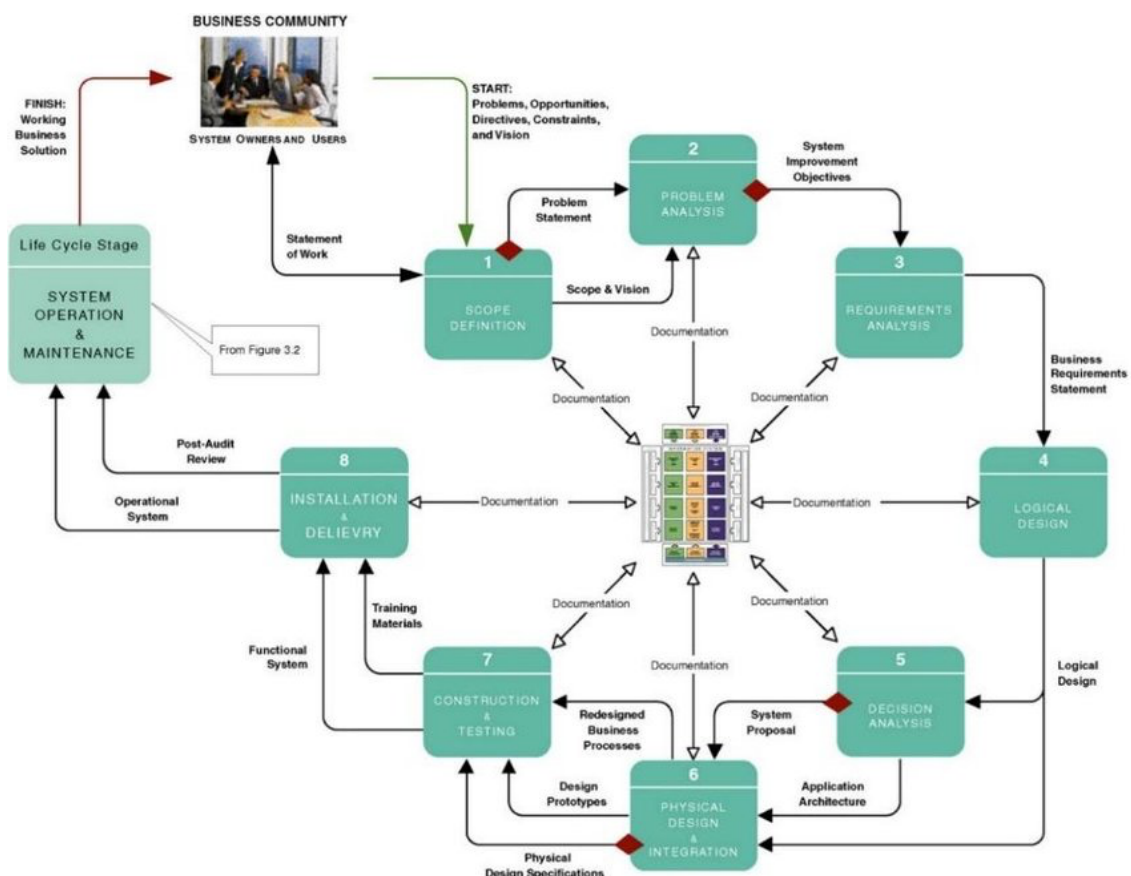
বাংলায়:

১. কী সমস্যা বা সুযোগ আছে?
২. এর সমাধানে কী করা উচিত?

Three Deliverables

1. **Project charter** – Project শুরু করার অনুমতি দেয়
2. **Problem statement** – কী সমস্যা তা ব্যাখ্যা করে
3. **Objective statement** – কী অর্জন করতে চায়, তা বলে

FAST Methodology



10. The requirements analysis phase is an essential part of a system development methodology.

According to the FAST methodology, which stakeholders typically participate in this phase? What is the primary focus of requirements analysis?

What is not the focus? How should each proposed requirement be evaluated? What critical error must be avoided?

Stakeholders Involved

- **System owners** (যারা সিস্টেমের জন্য টাকা দিচ্ছে বা মালিক)
- **System users** (যারা সিস্টেম ব্যবহার করবে)
- **System analysts** (যারা সিস্টেম তৈরি করবে)

Primary Focus

To identify **what the users need** from the system.

What is NOT the Focus?

The technical solution or system design.

Each requirement should be checked for:

- **Feasibility** (ব্যবহারযোগ্যতা আছে কিনা)
- **Clarity** (পরিষ্কারভাবে বলা আছে কিনা)
- **Testability** (পরীক্ষা করা সম্ভব কিনা)

Critical Error to Avoid

Don't skip user involvement.

If users are not involved, the system may fail to meet real needs.

11. In the FAST methodology, as well as most system methodologies, **system owners and system designers do not participate in the requirements analysis phase. What do you think the reason is for this?**

System owners and designers don't join the requirements analysis phase because:

1. **Avoid technical bias** – Designers might focus on how to build, not what users need.
2. **Keep focus on users** – This phase is for understanding user needs clearly.
3. **Clear roles** – Owners set goals earlier; this phase is for users and analysts.

১. প্রযুক্তিগত পক্ষপাত এড়িয়ে চলুন - ডিজাইনাররা ব্যবহারকারীদের কী প্রয়োজন তার উপর নয়, কীভাবে তৈরি করবেন তার উপর মনোযোগ দিতে পারেন।

২. ব্যবহারকারীদের উপর মনোযোগ দিন - এই পর্যায়টি ব্যবহারকারীর চাহিদা স্পষ্টভাবে বোঝার জন্য।

৩. স্পষ্ট ভূমিকা - মালিকরা আগে থেকেই লক্ষ্য নির্ধারণ করেন; এই পর্যায়টি ব্যবহারকারী এবং বিশ্লেষকদের জন্য।

12. What is the essential purpose of the logical design phase? How does it accomplish this? How are technological solutions incorporated in this phase? What are some common synonyms for this phase used by other methodologies? Who are the typical participants in this phase? What is agile modeling and what is its purpose? What are the deliverables coming out of this phase? In terms of the development team, what critical transition takes place by the end of this phase?

◆ Logical Design Phase (FAST Methodology)

- **Purpose:** System টা কি করবে, সেটা প্ল্যান করা, না যে কিভাবে করবে।
- **Focus:** Logical functions, data flow, and user interactions.
- **Technology use** না হয় এই phase এ।
- **Participants:** Analyst, users, subject experts.
- **Agile Modeling:** সহজভাবে diagram বানানো যাতে সবাই বুঝতে পারে।
- **Deliverables:**
 - Logical Data Model (ডাটা কীভাবে সংরক্ষণ হবে)
 - Process Model (কাজগুলো কিভাবে চলবে)
 - Interface Design (ব্যবহারকারী কিভাবে system ব্যবহার করবে)
- **Transition:** এই phase শেষ হলে team চলে যায় *Physical Design* phase এ — যেখানে system কিভাবে বানানো হবে সেটা decide হয়।

◆ Physical Design Phase (FAST Methodology)

Main Goal: System টা কিভাবে বানানো হবে—সেটা ঠিক করা।

Who must be involved:

👉 System designer আর developer – তারাই design করে।

Who may help:

👉 User আর analyst – তারা help বা feedback দিতে পারে।

◆ Two Types of Design:

1. **Packaged Design** – আগে থেকে বানানো software use করা (যেমন: SAP, MS Office)।
✓ Ready, fast, but কম control।
2. **Custom Design** – নতুন করে নিজের মত করে বানানো।
✓ More control, but time আর money বেশি লাগে।

14. A customer has engaged your software development company to develop a new order-processing system. However, the time frames are very tight and inflexible for delivery of at least the basic part of the new system. Further, user requirements are sketchy and unclear. What are two system development strategies that might be advantageous to use in this engagement? Amar kache important mone hoicheeee

1. Prototyping (Rapid Application Development - RAD)

- Build a quick **working model (prototype)** of the system.
- Get **user feedback** early and often.
- Refine the system step by step.
 - ✓ Good for unclear requirements.
 - ✓ Helps users see and improve early versions.

2. Agile Development

- Work in **small sprints** (short time-boxed phases).
- Involve users throughout the process.
- Adjust requirements as you go.
 - ✓ Flexible to change.
 - ✓ Fast delivery of a **basic working version (MVP)**.

15. potential downsides of using Prototyping and Agile strategies:

Amar kache important mone hoicheeee

Prototyping Downsides:

1. **Incomplete Design:** Final system may miss features.
2. **Too Many Changes:** Delays due to constant updates.
3. **Poor Documentation:** Lack of proper documentation.

Agile Downsides:

1. **Scope Creep:** Uncontrolled project growth.
2. **Requires User Involvement** – If users are not available regularly, progress may suffer.
3. **Not Ideal for Large Teams:** Works best with small teams.

✓ Prototyping এর সমস্যাগুলো:

1. অসম্পূর্ণ ডিজাইন – সব ফিচার নাও থাকতে পারে।
2. বেশি পরিবর্তন – বারবার পরিবর্তনে দেরি হয়।
3. দলিল কম – সঠিকভাবে ডকুমেন্টেশন হয় না।

✓ Agile এর সমস্যাগুলো:

1. স্কোপ ক্রিপ – কাজের পরিধি বেড়ে যায়।
2. ইউজার দরকার – ইউজার না থাকলে কাজ ধীর হয়।
3. বড় টিমে কঠিন – ছোট টিমে ভাল চলে।